Οργάνωση και Σχεδίαση Υπολογιστών (ECE 219)
Χειμερινό Εξάμηνο 2024-2025

Εργαστήριο 9

1. Εικονική Μνήμη (5 μονάδες)

Η εικονική μνήμη χρησιμοποιεί τον πίνακα σελίδων (page table) για να μεταφράσει εικονικές σε φυσικές διευθύνσεις και το TLB σαν cache για γρήγορη προσπέλαση του πίνακα σελίδων. Σε αυτήν την άσκηση θα δείξετε πως γίνεται η ενημέρωση του πίνακα σελίδων και του TLB γιά μια σειρά εικονικών διευθύνσεων όπως αυτές παράγονται από έναν επεξεργαστή. Υποθέστε ότι το σύστημα εικονικής μνήμης έχει σελίδες των 4 KB, μία fully associative TLB με 4 εισόδους και LRU πολιτική αντικατάστασης στο TLB.
Η παρακάτω σειρά από εικονικές διευθύνσεις (στο δεκαδικό σύστημα) θα χρησιμοποιηθεί στα επόμενα ερωτήματα της άσκησης.
9452, 30964, 19136, 46502, 38110, 16653, 48480.
Η αρχική κατάσταση του TLB:

Valid Bit	Tag	PP number
1	11	12
1	7	4
1	3	6
0	4	9

Η αρχική κατάσταση του πίνακα σελίδων:

Valid Bit	PP number
1	5
0	Δίσκος
0	Δίσκος
1	6
1	9
1	11
0	Δίσκος
1	4
0	Δίσκος
0	Δίσκος
1	3
1	12

Για τον αλγόριθμο LRU στο TLB, θεωρείστε ότι αρχικά οι λιγώτερο πρόσφατες είσοδοι είναι κατά σειρά οι #0, #1, και #2.

a) Να δείξετε τι ακριβώς συμβαίνει στον πίνακα σελίδων και στο TLB γιά κάθε προσπέλαση στις παραπάνω διευθύνσεις. Ποιά είναι η τελική κατάσταση του πίνακα σελίδων και του TLB;
b) Επαναλάβετε την άσκηση (a) χρησιμοποιώντας σελίδες μεγέθους 16 KB (αντί για 4 KB). Να εξηγήσετε ποιά τα πλεονεκτήματα και ποιά τα μειονεκτήματα του μεγαλύτερου μεγέθους σελίδας.
c) Επαναλάβετε την άσκηση (a) χρησιμοποιώντας direct mapped TLB (4 KB σελίδας). Να συζητήσετε την σημασία ύπαρξης ενός TLB υψηλής απόδοσης. Πως θα γινόταν ο χειρισμός τηης εικονικής μνήμης χωρίς την ύπαρξη του TLB;

Υπάρχουν διάφορες παράμετροι που επηρεάζουν το συνολικό μέγεθος του πίνακα σελίδων. Έστω ότι στο σύστημα μας, υπάρχουν οι παρακάτω παράμετροι:
Μέγεθος Εικονικής Διεύθυνσης : 64 bits
Μέγεθος Σελίδας : 16 KB
Μέγεθος κάθε entry (εισόδου) στον πίνακα σελίδων : 8 bytes

d) Με βάση τις παραπάνω παραμέτρους να υπολογίσετε το συνολικό μέγεθος του πίνακα σελίδων (σε bytes) σε ένα σύστημα που εκτελεί ταυτόχρονα 5 εφαρμογές.
e) Να υπολογίσετε υπολογίσετε το συνολικό μέγεθος του πίνακα σελίδων (σε bytes) σε ένα σύστημα που χρησιμοποιεί πίνακα σελίδων δύο επιπέδων και που εκτελεί ταυτόχρονα 5 εφαρμογές. Το πρώτο επίπεδο έχει 256 entries και κάθε entry του έχει μέγεθος 8 bytes. Για να καταλάβετε τι εννοώ με τον όρο "πίνακας σελίδας δύο επιπέδων", κοιτάξτε τις διαφάνειες #22, #23 της διάλεξης για την εικονική μνήμη.
f) Θέλουμε να σχεδιάσουμε μιά μνήμη cache που χρησιμοποιεί την τεχνική virtually indexed and physically tagged. Με το μέγεθος σελίδας που δείχνουν οι παραπάνω παράμετροι, μπορούμε να κατασκευάσουμε μιά direct mapped 16 KB cache, υποθέτοντας 2 words σε κάθε cache block; Επίσης, πως μπορούμε να αυξήσουμε το μέγεθος της cache πάνω από 16 KB;

2. Απόδοση Ιεραρχίας Μνήμης (5 μονάδες)

Θεωρήστε έναν επεξεργαστή με σύστημα ιεραρχίας εικονικής μνήμης (virtual memory) 64-bits που αποτελείται από μνήμη L1 D-Cache ενός επιπέδου και κύρια μνήμη. Η φυσική διεύθυνση βρίσκεται με τη βοήθεια μιας μνήμης TLB, η οποία διατηρεί τις πιο πρόσφατες απεικονίσεις από εικονικές σε φυσικές διευθύνσεις, και ενός πίνακα σελίδων (page table) που χρησιμοποιείται για την εύρεση των φυσικών διευθύνσεων από τις εικονικές. Η μνήμη L1 D-Cache έχει μέγεθος 16 KB, άμεσης απεικόνισης (direct mapped), και μέγεθος block 32 bytes.

Το μέγεθος της εικονικής σελίδας είναι 4 KB. Κάθε φορά που συμβαίνει ένα σφάλμα σελίδας (page fault), το λειτουργικό σύστημα αναλαμβάνει να φέρει από τον σκληρό δίσκο όχι μόνο την σελίδα που δημιούργησε το σφάλμα, αλλά και την αμέσως επόμενη σελίδα σε περίπτωση που χρειαστεί στο μέλλον (prefetch). Μόνο η απεικόνιση της ζητούμενης σελίδας εισέρχεται στο TLB. Δεν υπάρχει δηλ. prefetching στο TLB, αλλά μόνο στην κύρια μνήμη και στον πίνακα σελίδων.

Έστω ότι ο κώδικας C που δείχνουμε στο παρακάτω σχήμα εκτελείται στον επεξεργαστή μας. Υποθέστε ότι ένας double καταλαμβάνει 8 bytes, και ότι η εντολή malloc δεσμεύει διαδοχικές θέσεις στην μνήμη (στο heap) για να αποθηκεύσει την δυναμικά διασυνδεδεμένη λίστα. Η μεταβλητή sum τοποθετείται από τον compiler σε κάποιον καταχωρητή.

a) Να υπολογίσετε τον συνολικό αριθμό των αστοχιών στο TLB και των αστοχιών στην L1 D-Cache. Υποθέστε ότι το TLB και η L1 D-Cache είναι αρχικά κενές.

b) Να υπολογίσετε το miss rate του TLB και της L1 D-Cache;

c) Υποθέστε ότι ο χρόνος προσπέλασης τόσο της L1 D-Cache όσο και του TLB είναι 1 κύκλος μηχανής. Ο χρόνος προσπέλασης της κύριας μνήμης για οποιονδήποτε λόγο είναι 50 κύκλοι μηχανής ενώ ο χρόνος προσπέλασης του σκληρού δίσκου σε περίπτωση σφάλματος σελίδας (page fault) είναι 10.000 κύκλοι. Να υπολογίσετε τον Μέσο Χρόνο Προσπέλασης στην Μνήμη (AMAT) θεωρώντας ότι οι μόνες προσπελάσεις στην μνήμη γίνονται μέσα στο while loop. Αγνοείστε εντελώς την L1 I-Cache.

   Υπόδειξη: θεωρείστε όλες τις δυνατές περιπτώσεις για το αποτέλεσμα μιας προσπέλασης της μνήμης.

```
#define N 256

struct element {
    double A[127];
    struct element *next;
};

double sum = 0;
struct element *node = malloc(N * sizeof(element));

node = create_list(N);

while (node != null) {
   for (int i = 0 ; i < 127 ; i ++) {
    sum += node->A[i];
   }
   node = node->next;
}
```